

Chương 4 đã trình bày toàn diện quá trình thiết kế, triển khai, kiểm thử và vận hành hệ thống giám sát chất lượng không khí. Trong quá trình đó, một số quyết định kỹ thuật đòi hỏi phân tích sâu và giải quyết những thách thức không tầm thường. Chương 5 tập trung phân tích bốn đóng góp kỹ thuật nổi bật nhất, mỗi đóng góp được trình bày theo cấu trúc ba phần: (i) bài toán và thách thức đặt ra, (ii) giải pháp được lựa chọn và thiết kế, (iii) kết quả đạt được. Bốn đóng góp bao gồm: xây dựng hệ thống Internet vạn vật (Internet of Things – IoT) đa tầng end-to-end (phần 0.1), thiết kế giao thức Long Range (LoRa) binary và cơ chế chống trùng lặp dữ liệu (phần 0.2), kiến trúc Backend phân lớp và pipeline xử lý telemetry (phần 0.3), và hệ thống chi phí thấp với mã nguồn mở (phần 0.4).

0.1 Xây dựng hệ thống IoT đa tầng end-to-end

0.1.1 Bài toán và thách thức

Hệ thống giám sát chất lượng không khí không phải là một ứng dụng đơn lớp mà là sự tích hợp của bốn miền kỹ thuật hoàn toàn khác biệt: thiết kế và lắp ráp phần cứng, lập trình nhúng trên vi điều khiển (C/C++ với PlatformIO), phát triển backend phía máy chủ (Node.js/Express), và phát triển giao diện web phía người dùng (React/Vite). Mỗi miền yêu cầu bộ công cụ, ngôn ngữ lập trình, và phương pháp debug riêng biệt, tạo ra thách thức tích hợp đặc thù: các lỗi ở ranh giới giữa hai tầng thường chỉ biểu hiện khi toàn bộ hệ thống chạy end-to-end, khó tái hiện trong môi trường kiểm thử riêng lẻ từng tầng.

Cụ thể, giao tiếp giữa các tầng yêu cầu tính đồng nhất tuyệt đối về format dữ liệu. Gói tin LoRa 18 byte do firmware Sensor Node tạo ra phải được Gateway giải mã chính xác theo cùng định nghĩa struct. Batch JSON mà Gateway gửi lên Backend phải khớp với schema validation của Controller. Sự kiện Socket.IO mà Backend phát đi phải được Frontend xử lý đúng theo cùng tên event và cấu trúc payload. Bất kỳ sự không nhất quán nào giữa hai tầng liên kế đều dẫn đến dữ liệu sai hoặc hệ thống ngừng hoạt động.

0.1.2 Giải pháp

Để kiểm soát độ phức tạp tích hợp, em áp dụng chiến lược *contract-first*: định nghĩa đầy đủ giao thức và schema giao tiếp giữa các tầng trước khi viết mã nguồn từng tầng. Giao thức LoRa binary 18 byte được định nghĩa dưới dạng struct C và tải liệu hóa đầy đủ trước khi triển khai cả firmware lẫn parser phía Backend. Schema JSON của HTTP POST từ Gateway được cố định ngay từ đầu để cả hai phía phát triển độc lập mà vẫn tương thích. Tên và payload của các Socket.IO event được thống nhất giữa Backend và Frontend trước khi triển khai.

Chiến lược phát triển theo thứ tự từ dưới lên cũng giúp kiểm soát rủi ro: firmware

Sensor Node được hoàn thiện và kiểm thử trước, tiếp đến là firmware Gateway, rồi Backend, và cuối cùng là Frontend. Mỗi tầng được kiểm thử độc lập trong phạm vi có thể (ví dụ, Backend nhận dữ liệu giả lập từ Postman để kiểm thử trước khi có Gateway thật), giúp khoanh vùng lỗi nhanh hơn. Docker Compose đảm bảo môi trường chạy Backend và cơ sở dữ liệu nhất quán giữa máy phát triển và máy chủ production, loại trừ sai sót do cấu hình môi trường.

0.1.3 Kết quả

Hệ thống hoàn chỉnh bốn tầng được đưa vào vận hành production trong khoảng bốn tháng phát triển, với tổng quy mô mã nguồn khoảng 13.759 dòng mã trải qua bốn nhóm công nghệ (1.262 LOC firmware Sensor Node C/C++, 1.608 LOC firmware Gateway C/C++, 2.732 LOC Backend Node.js, 8.050 LOC Frontend React, cùng 384 LOC SQL và cấu hình triển khai). Toàn bộ chín chức năng và năm yêu cầu phi chức năng được hiện thực đầy đủ (Bảng ??). Quy trình triển khai được rút gọn xuống còn một lệnh duy nhất (`docker compose up`), với tính năng tự đăng ký Gateway và Sensor Node không cần thao tác cấu hình thủ công trên máy chủ.

0.2 Thiết kế giao thức LoRa binary và cơ chế chống trùng lặp dữ liệu

0.2.1 Bài toán và thách thức

Truyền thông LoRa trong hệ thống đặt ra hai thách thức đặc thù. Thứ nhất, LoRa là giao thức truyền một chiều không có cơ chế xác nhận (ACK) ở tầng ứng dụng: Sensor Node phát gói tin nhưng không biết liệu Gateway có nhận được hay không. Cùng với đó, trong các kịch bản triển khai mật độ cao, vùng phủ sóng của hai Gateway có thể chồng lấp nhau, dẫn đến cùng một gói tin từ Sensor Node được nhiều Gateway thu nhận đồng thời và chuyển lên Backend — gây trùng lặp dữ liệu trong cơ sở dữ liệu nếu không có cơ chế lọc.

Thứ hai, LoRa là kênh truyền băng thông hẹp với air-time (thời gian chiếm dụng kênh truyền) tỷ lệ thuận với kích thước gói tin. Nếu sử dụng định dạng JSON cho gói tin LoRa (tương tự phần Gateway–Backend), mỗi gói tin sẽ có kích thước khoảng 150–200 byte, kéo dài đáng kể air-time và tăng nguy cơ va chạm gói tin trong môi trường nhiễu thiết bị.

0.2.2 Giải pháp

Để giải quyết vấn đề kích thước gói tin, giao thức LoRa được thiết kế dưới dạng nhị phân (binary protocol) với cấu trúc struct C packed (không có padding byte) tổng kích thước 18 byte. Gói tin gồm ba trường tiêu đề: `nodeId` (1 byte, định danh node), `pktType` (1 byte, phân loại gói dữ liệu hoặc heartbeat), `msgId` (1 byte, bộ đếm tuần hoàn 0–255). Bảy trường payload lưu các giá trị cảm biến dưới dạng số

nguyên: PM1, PM2.5, PM10 (mỗi trường 2 byte, nhân 10 để giữ một chữ số thập phân), CO₂ và TVOC (mỗi trường 2 byte), nhiệt độ (2 byte có dấu, nhân 10), độ ẩm (2 byte, nhân 10), và battery (1 byte, 0–100%). Biến đổi nhân 10 loại bỏ hoàn toàn số dấu phẩy động khỏi gói tin, đồng thời vẫn giữ đủ độ chính xác cho hiển thị. Giá trị sentinel lần lượt là 0xFFFF (unsigned) và 0x7FFF (signed) đánh dấu cảm biến bị lỗi đọc. Cấu trúc gói tin được minh họa tại Hình ???. So với JSON, kích thước 18 byte giảm air-time khoảng 11 lần.

Để giải quyết vấn đề trùng lặp dữ liệu, Backend triển khai cơ chế deduplication dựa trên trường msgId. Mỗi Sensor Node duy trì bộ đếm tăng dần tuần hoàn; khi Backend nhận gói tin, module dedupCache kiểm tra cặp khóa (nodeId, msgId) trong bộ nhớ đệm trước khi ghi vào cơ sở dữ liệu. Nếu cặp khóa đã tồn tại, gói tin được nhận diện là bản sao và bị loại bỏ. Chi tiết triển khai được trình bày tại phần ??.

0.2.3 Kết quả

Giao thức binary 18 byte giảm air-time LoRa xuống khoảng 11 lần so với phương án JSON tương đương, cải thiện đáng kể thông lượng kênh truyền và giảm tiêu thụ điện của module LoRa (thời gian phát sóng ngắn hơn). Cơ chế deduplication hoạt động chính xác trong toàn bộ các kịch bản kiểm thử: dữ liệu chỉ được ghi một lần duy nhất ngay cả khi nhiều Gateway chuyển tiếp cùng một gói tin. Thử nghiệm khoảng cách truyền LoRa trong không gian thoáng đạt 800 mét (Bảng ??), đủ để một Gateway đặt tại vị trí trung tâm phục vụ nhiều Sensor Node trong phạm vi bán kính triển khai thực tế.

0.3 Kiến trúc Backend phân lớp và pipeline xử lý telemetry

0.3.1 Bài toán và thách thức

Backend phải đáp ứng đồng thời bốn luồng xử lý có mức độ ưu tiên khác nhau: tiếp nhận và lưu dữ liệu telemetry từ Gateway (quan trọng nhất — không được mất dữ liệu), kiểm tra ngưỡng và tạo cảnh báo, phát sự kiện realtime qua Socket.IO, và phục vụ các REST API cho Frontend. Thách thức thiết kế then chốt là đảm bảo tính bền vững của dữ liệu: nếu bước kiểm tra cảnh báo xảy ra lỗi (ví dụ, lỗi truy vấn bảng cấu hình), logic lưu dữ liệu đo không được bị cuốn theo rollback. Trước khi thiết kế lại, cách tiếp cận đơn giản nhất là đặt toàn bộ xử lý vào một transaction — nhưng điều này vi phạm tính độc lập giữa hai nghiệp vụ.

Một thách thức khác là hiệu năng truy vấn dữ liệu lịch sử. Bảng measurements tăng trưởng liên tục (mỗi Sensor Node ghi một bản ghi mỗi 30 phút); truy vấn thống kê theo giờ cho dài 30 ngày yêu cầu GROUP BY trên hàng chục nghìn bản ghi, dễ dẫn đến thời gian phản hồi vượt ngưỡng 3 giây yêu cầu phi chức năng.

0.3.2 Giải pháp

Pipeline xử lý telemetry được thiết kế gồm ba bước tách biệt. Bước một, module `dedupCache` lọc gói tin trùng lặp trước khi bất kỳ thao tác cơ sở dữ liệu nào diễn ra. Bước hai, một database transaction duy nhất cập nhật trạng thái Gateway, cập nhật trạng thái Sensor Node và ghi bản ghi đo vào bảng `measurements` — ba thao tác này được gộp chung trong một transaction để đảm bảo tính nhất quán. Bước ba, sau khi transaction kết thúc thành công, module `alertService` kiểm tra ngưỡng và tạo cảnh báo nằm *ngoài* transaction, đảm bảo lỗi cảnh báo không ảnh hưởng đến dữ liệu đo vừa được ghi. Biểu đồ trình tự mô tả pipeline này được trình bày tại Hình ???. Luồng phụ thuộc giữa các module tuân thủ nguyên tắc một chiều: `routes` → `controllers` → `services` → `config`, đảm bảo khả năng kiểm thử và bảo trì từng lớp độc lập.

Để giải quyết hiệu năng truy vấn lịch sử, hệ thống tận dụng tính năng Continuous Aggregate của TimescaleDB. View `hourly_measurements` được TimescaleDB tự động cập nhật gia tăng (incremental refresh) mỗi khi có dữ liệu mới ghi vào bảng `measurements`, lưu sẵn giá trị trung bình theo giờ cho từng Sensor Node. Khi Frontend yêu cầu dữ liệu lịch sử theo chế độ giờ, Backend truy vấn trực tiếp từ view tập hợp sẵn thay vì thực hiện GROUP BY tốn kém trên bảng gốc. Chi tiết thiết kế cơ sở dữ liệu được trình bày tại phần ??.

Ngoài ra, module `alertService` triển khai cơ chế cooldown 15 phút: nếu cùng một Sensor Node và cùng một thông số đã phát sinh cảnh báo trong 15 phút gần nhất, cảnh báo mới sẽ không được tạo, ngăn chặn tình trạng spam thông báo khi dữ liệu ô nhiễm liên tục vượt ngưỡng trong thời gian ngắn.

0.3.3 Kết quả

Pipeline ba bước đảm bảo zero data loss: trong toàn bộ quá trình kiểm thử và vận hành, không có bản ghi đo nào bị mất do lỗi ở bước kiểm tra cảnh báo. Module tính AQI (`aqiService`) — nghiệp vụ cốt lõi của hệ thống — được kiểm thử đơn vị với 19 test case tập trung vào giá trị biên, tất cả đều pass (phần ??). Tổng cộng 44/44 kịch bản kiểm thử (25 thủ công và 19 tự động) đạt kết quả pass (phần ??). Thời gian phản hồi API truy vấn dữ liệu 30 ngày nhờ Continuous Aggregate luôn dưới 3 giây, đáp ứng đúng yêu cầu phi chức năng. Hệ thống duy trì uptime 100% trong 7 ngày theo dõi liên tục trên môi trường production.

0.4 Hệ thống chi phí thấp với mã nguồn mở hoàn toàn

0.4.1 Bài toán và thách thức

Một hệ thống giám sát chất lượng không khí chỉ có giá trị thực tiễn khi có thể triển khai với mật độ đủ dày để phản ánh chính xác chất lượng không khí của một khu vực. Các giải pháp thương mại hiện có như IQAir (khoảng 10.000.000 VNĐ/thiết bị) hay các trạm đo chuẩn của VN-AQI (hàng chục đến hàng trăm triệu đồng/trạm) có chi phí quá cao để triển khai diện rộng, đặc biệt trong các khu dân cư hoặc trường học. Ngoài ra, tất cả các giải pháp thương mại khảo sát ở Chương 2 đều là hệ thống đóng (closed-source), không cho phép tùy biến logic nghiệp vụ hoặc tích hợp với hệ thống bên thứ ba.

0.4.2 Giải pháp

Hệ thống được xây dựng hoàn toàn từ linh kiện phổ thông, dễ mua, có tài liệu cộng đồng phong phú. Sensor Node sử dụng ESP32 DevKit V1 làm vi điều khiển trung tâm, kết hợp module LoRa AS32-TTL-100 và ba cảm biến: PMS7003 (bụi mịn PM1/PM2.5/PM10), CCS811 (CO₂ và TVOC), AHT10 (nhiệt độ và độ ẩm). Gateway sử dụng cùng board ESP32 với module LoRa tương tự. Đây đều là linh kiện có thể mua tại các cửa hàng điện tử trong nước, không phụ thuộc vào nhà cung cấp chuyên biệt.

Toàn bộ mã nguồn hệ thống — bao gồm firmware C/C++ cho Sensor Node và Gateway, Backend Node.js/Express, và Frontend React — được công khai dưới giấy phép mã nguồn mở, cho phép bên thứ ba tái sử dụng, tùy biến và mở rộng. Hệ thống hỗ trợ thêm Sensor Node và Gateway mới thông qua giao diện web captive portal (provisioning không dây) mà không cần truy cập vật lý vào thiết bị hay can thiệp vào cấu hình máy chủ. So sánh chi tiết với các hệ thống hiện có được trình bày tại Bảng ??.

0.4.3 Kết quả

Chi phí linh kiện cho một Sensor Node hoàn chỉnh là khoảng 895.000 VNĐ (36 USD), thấp hơn giải pháp IQAir khoảng 11 lần (Bảng ??). Áp dụng mô hình Hub-Spoke, một Gateway (310.000 VNĐ) có thể phục vụ nhiều Sensor Node trong bán kính 500–800 mét, khiến chi phí bình quân mỗi điểm đo tiếp tục giảm khi quy mô triển khai tăng lên. Hệ thống đề xuất là giải pháp duy nhất trong số các hệ thống khảo sát kết hợp đồng thời bốn điểm khác biệt: đo đa thông số (sáu thông số), truyền thông LoRa không cần WiFi tại từng trạm, mã nguồn mở hoàn toàn, và chi phí phần cứng thấp.

Chương 5 đã phân tích bốn đóng góp kỹ thuật nổi bật của hệ thống. Tổng hợp

lại, các đóng góp thể hiện ba ý nghĩa cốt lõi. Thứ nhất, về mặt tích hợp, việc phối hợp bốn miền kỹ thuật khác nhau thành một hệ thống vận hành ổn định chứng minh tính khả thi của quy trình phát triển contract-first kết hợp kiểm thử theo từng tầng. Thứ hai, về mặt kỹ thuật, hệ thống giải quyết thành công các thách thức đặc thù của IoT trong điều kiện thực tế: thiết kế giao thức truyền thông tinh gọn với khả năng chống lỗi dữ liệu, và kiến trúc Backend đảm bảo tính bền vững của dữ liệu đo. Thứ ba, về mặt thực tiễn, chi phí thấp và mã nguồn mở tạo ra điều kiện để hệ thống được nhân rộng và tùy biến cho các ngữ cảnh triển khai khác nhau. Chương 6 sẽ tổng kết toàn bộ đồ án và đề xuất các hướng phát triển trong tương lai.